SC2006 Software Engineering

# Lab 4 Deliverables: Coding Agent Report

| | |
|---|---|
| **Lab group** | SCSL, Group 4 |
| **Team** | CleanCravings |
| **Members** | Rafael, Albin, Samrithi, Jovan, Jun Ren |
| **Author(s) of this document** | Rafael, Albin |
| **Coding agent** | Claude Code 2.1.306 (Anthropic's coding agent, used from the terminal), model Claude Opus 5.5 |
| **Date of the experiments** | 5 November 2026 |

## Contents


1. How we ran the experiments
2. Exercise 3.3.2: regenerating a deleted control class (HygieneController)
3. Exercise 3.3.3: code quality review and refactoring
4. Appendix: prompts, the agent's replies, its code and its changes

# 1. How we ran the experiments

The lab suggests an in-IDE agent such as Cline. We used **Claude Code**, a comparable coding agent that reads and edits the files of a project itself. For each experiment:

- We exported a **fresh copy** of our repository into an empty folder and started a **new agent session** there, so the agent had no memory of earlier work and could not see our real working copy. We removed `CLAUDE.md` (our notes for AI tools) from the copy, because it describes HygieneController's rules in prose and would have given the answer away.
- The agent could **read, search, create and edit files** in the copy, but **could not run commands**. We ran the test suite ourselves afterwards, so we could see exactly what it changed and judge it against the same tests we use.
- We compared its files with the originals (`diff`) and ran our full backend test suite (pytest), which includes the Lab 4 black-box and basis path tests.

The exact prompts and the agent's replies are in the appendix. The files it produced are kept in `docs/lab4/agent_experiments/` in our repository.

# 2. Exercise 3.3.2: regenerating HygieneController

### What we did

We chose **HygieneController**, the control class we black-box tested (section 1 of our Lab 4 test cases). We deleted the whole file, `backend/app/controllers/hygiene_controller.py`; with it gone, the test suite could not even start (an import error during collection). We then gave the agent one prompt (appendix A.1) telling it the file had been deleted and pointing it to:

- our tests, especially the Lab 4 black-box tests of `passes_threshold`;
- the code that calls HygieneController;
- our design class diagrams and the requirements cited in code comments (FR-2.1 to FR-2.5).

We did not give it the original class, not even with the function bodies removed.

### Result

| | |
|---|---|
| **Iterations needed** | None: one prompt, no follow-up |
| **Agent's work** | About 1 minute 45 seconds; 18 file reads and searches, then one file written |
| **Other files changed** | None (checked with `diff`) |
| **Test suite on its class** | **127 passed, 1 skipped, 1 warning in 6.07s**, the same as with our original class |

The agent rebuilt the class with the same public interface as ours: the `GRADE_RANK` ordering A > B > NEW > C and the four methods `current_grade`, `passes_threshold`, `summary` and `track_record`, with the same signatures. It worked these out from the callers and the class diagrams, and the rules from the tests: for example, that a stall with no current grade fails any minimum, and that a snapshot older than the latest SFA export means the grade is no longer current. It added docstrings citing the requirements, and sorted the grade history by date instead of relying on the database's ordering.

Comparing it with our original, line by line, showed **two differences in behaviour that none of our tests detected**:

1. `summary()` **counts only Cooked Food stalls**; ours counted every stall passed to it. Our SFA import only keeps Cooked Food stalls (all 5,614 stalls in our data are), so this makes no visible difference today; the agent's version is the more defensive one (FR-1.6).

2. `track_record()` **lists every grade snapshot**; ours collapses consecutive identical snapshots so the history shows only changes. Every licence has one snapshot so far, so the two look the same now, but after a few months of daily SFA imports the agent's version would show dozens of identical "A" rows on a stall page. This would be a real regression, and our tests only check that the history is not empty.

It also turned our comment "Team to confirm this choice" about where NEW ranks into a settled statement citing FR-2.2.

A second run of the same prompt happened by accident after the first had finished. That session found the agent's own new file, took it for the original and refused to overwrite it, saying a rewrite after reading it would not be a valid "from scratch" result (appendix A.3). It did not change anything.

### Reflection

The outcome exceeded our expectations: the agent rebuilt a working class in one attempt, from tests and diagrams alone, without seeing the original or running the tests. Its explanation was accurate and traced each decision to a test or a requirement. The experiment also showed how much it relied on our tests to define "correct": where a behaviour was only in our code and not in a test or a diagram (collapsing the grade history), it reasonably chose something else, and the suite still passed. The lesson for us is that our tests are the real specification when an AI writes code, so the gap to close is a test for the collapsed grade history. Had the agent failed, the next steps we had planned were to paste the test failures back to it, then to give it the original file with only the function bodies removed.

## 3. Exercise 3.3.3: code quality review and refactoring

### What we did

Our further experiment was the lab's third suggestion: ask the agent for its opinion on the quality of our codebase, and have it carry out one of its refactoring suggestions. In a new fresh copy and a new session, we asked it (appendix B.1) to rank the five refactorings that would most improve the backend, with reasons and risks, then to carry out only the top one without changing behaviour or tests, and to explain how it made sure behaviour was unchanged.

### Result

| | |
|---|---|
| **Iterations needed** | None: one prompt, no follow-up |
| **Agent's work** | About 3 minutes 10 seconds over 43 steps of reading and editing |
| **Files changed** | 4 controller files, 48 changed lines; no tests changed |
| **Test suite before and after** | 127 passed, 1 skipped, 1 warning in 5.93s after, the same as before |
| **Our extra check** | The original and refactored centre list and recommendations give **identical results on our real data in all 15 scenarios we tried** (every minimum grade, with and without the halal restriction) |

**Its five suggestions** (in full in appendix B.2):

1. define each stall-filtering rule once;
2. share the centre query and the Cooked Food filter;
3. split up the 100-line `recommend()` method;
4. move business logic out of the API router;
5. cut the per-row database queries in the SFA import.

Each came with a specific reason and a sensible risk. For example, for number 5 it warned that a licence appearing twice in one export would behave differently, and said to add a test for that first.

**The refactoring it carried out** (number 1) was well chosen:

- **One rule instead of three.** Our minimum-grade rule (FR-2.5) was written out in three places: `passes_threshold`, the cached centre list and the recommendations. The halal check and the suspension flag were each written twice. It added `HygieneController.grade_passes()` and `suspended_past_year()` and made every caller use them.
- **It caught an error in our own documentation.** Our Lab 4 test document says the centre list and recommendations use `passes_threshold()`. They had kept their own copies of the rule since we added caching, so our 16 black-box tests only covered one of the three copies.
- **It named a remaining gap.** No test checks which centres the minimum-grade filter returns.

### Reflection

This met and in places exceeded our expectations. The review was specific to our code rather than generic advice, the ranking was reasonable, and the refactoring was small, safe and easy to check. Its "proof" that behaviour was unchanged was careful reasoning, not a test run, since it could not run commands; we confirmed it with the test suite and the real-data comparison. The most valuable result was finding the mismatch between our test document and our code, which a human review had missed. Adopting this refactoring would make the rule our black-box tests cover the one every list uses, and the missing centre-list test would close the gap it found. Its suggestions 3 and 5 are medium risk, and we would only attempt them with new tests in place first, as it advised.

# Appendix

## A.1 Exercise 3.3.2: our prompt

> This is our SC2006 group project, CleanCravings (FastAPI backend in backend/, React frontend in frontend/). The file backend/app/controllers/hygiene_controller.py, which contained our control class HygieneController, has been deleted. Please recreate it from scratch.
>
> To work out what it must do, use:
> - the tests in backend/tests/, especially tests/test_lab4.py (black-box tests of HygieneController.passes_threshold) and tests/test_api.py;
> - the rest of the codebase, in particular the places that import and call HygieneController (backend/app/controllers/);
> - our design class diagrams in docs/diagrams/ (design_class_app.puml and overall_class.puml) and the requirements cited in the code comments (FR-2.1 to FR-2.5).
>
> Only create backend/app/controllers/hygiene_controller.py; do not change any other file. When you are done, briefly explain your design decisions.

## A.2 The agent's reply

I've recreated `backend/app/controllers/hygiene_controller.py` and changed no other file. I couldn't run the tests because there's no shell in this session. I checked the file by hand against all 16 black-box cases in `test_lab4.py` and against `test_etl.py`, `test_api.py` and the three callers. Run `pytest` in `backend/` to confirm.

**What the class provides.** The callers and both class diagrams use a module-level `GRADE_RANK` and four static methods: `current_grade`, `passes_threshold`, `summary` and `track_record`. The class has those and nothing more.

**Design decisions:**

1. **Grade order A=4, B=3, NEW=2, C=1 (FR-2.2).** NEW sits above C so it never looks like a failing grade, which tests BB-07 and BB-08 check. All ranks are above 0 because `HawkerCentreController._cached_summaries` uses `default=0` to mean "no graded stall".
2. `current_grade(licensee)` **(FR-2.1).** It takes the newest snapshot, sorted by date. It doesn't rely on the database's ordering because the Lab 4 tests build objects in memory. If that snapshot is older than `last_seen_snapshot`, it returns `None`: the latest SFA export listed the licence without a grade, such as "Not Under SAFE". Tests BB-14, BB-15 and the ETL "Not Under SAFE" test cover this.
3. `passes_threshold(stall, threshold)` **(FR-2.5, FR-5.4).** With no minimum set, every stall passes. With a minimum, a stall passes only if its grade ranks at or above it. A stall with no current grade or no linked licensee fails. This matches the rule `RecommendationController` already copies inline. Invalid minimums such as "D" or "a" are rejected by the API (HTTP 422) before they reach this method, so it doesn't check for them.
4. `summary(stalls)`**.** It counts only Cooked Food stalls (FR-1.6), because `presenters.centre_summary` passes it all of a centre's stalls. Stalls with no current grade are counted as `ungraded`.
5. `track_record(stall)` **(FR-2.3, FR-2.4).** It's built from the snapshots rather than stored, as the entity diagram notes.
   - **Suspended flag:** it comes from `licence_status`, which is what the ETL test needs and what `stall_summary` uses. The grade row can't be used because a "Not Under SAFE" export creates no grade row.
   - **Last updated:** this is the date of the latest SFA export that listed the licence.
   - **Grade history:** all grades, newest first.
   - **Demerit points:** this stays null because SFA doesn't publish them.
   - **No licensee:** it returns an empty record instead of failing.

## A.3 The accidental second run's reply

I haven't recreated the file, because it hasn't been deleted. `backend/app/controllers/hygiene_controller.py` still exists with a complete 69-line implementation of `HygieneController`, so I stopped instead of overwriting it.

**The existing file matches everything you pointed me to.** I checked it by reading only; I couldn't run pytest here.

- **Design class diagram** (`design_class_app.puml:111`): it has `GRADE_RANK` (A > B > NEW > C), `current_grade`, `passes_threshold`, `summary` and `track_record`, with the same names and signatures.
- **Callers:** `hawker_centre_controller.py` imports `GRADE_RANK` and `HygieneController` and uses `0` to mean "no current grade", which works because every rank is above 0. `presenters.py` uses `current_grade` and `summary`, and `recommendation_controller.py` uses `GRADE_RANK`. All of these exist.

- **test_lab4.py BB-01 to BB-16:** with no minimum, every stall passes. A stall with no grade or no licensee fails any minimum. The newest snapshot counts as the current grade only if it's dated on or after `last_seen_snapshot`. All 16 cases agree with the code.
- **test_etl.py:** "Not Under SAFE" clears the current grade, and `track_record` reports suspension, `last_updated` and the grade history. The code handles all of these.

**Why I didn't overwrite it:** a rewrite now wouldn't really be "from scratch", because I've already read the original. It would also mean replacing working code that matches your design with my own version for no benefit. If this is an exercise to see whether the class can be rebuilt from the tests, diagrams and requirements alone, the deletion didn't happen and the result wouldn't be valid.

How would you like to continue?

1. **Keep the existing file.** You don't need to do anything, and you can run `pytest backend/tests` to confirm it passes.
2. **Delete it yourself and start a fresh session.** I can then rebuild it from the tests, diagrams and FR-2.1 to FR-2.5 without having seen the original.
3. **Have me rewrite it here anyway.** I'll do that if you want, but the result will be shaped by the version I've already read.

## A.4 The class the agent wrote

```
"""HygieneController (FR-2.1 to FR-2.5).

Works out a stall's current SAFE grade from the dated SFA snapshots, its track record,
and whether it meets a user's Hygiene Threshold (also used for FR-5.4)."""

from collections.abc import Iterable

from ..models import Licensee, SAFEGrade, SafeGradeValue, Stall, StallType
from ..schemas import GradeSummary, SafeGradeOut, TrackRecordOut

# A > B > NEW > C. NEW (licensed under a year) ranks above C because SFA says it
# must not be presented as a failing grade (FR-2.2). All ranks are > 0, so callers
# can use 0 for "no current grade".
GRADE_RANK = {SafeGradeValue.A: 4, SafeGradeValue.B: 3, SafeGradeValue.NEW: 2, SafeGradeValue.C: 1}


def _history(licensee: Licensee) -> list[SAFEGrade]:
    # Newest first. The relationship is already ordered that way when loaded from the
    # database, but objects built in memory may not be
    return sorted(licensee.grades, key=lambda g: g.snapshot_date, reverse=True)


class HygieneController:
    @staticmethod
    def current_grade(licensee: Licensee | None) -> SAFEGrade | None:
        """The newest snapshot (FR-2.1), but only if it came from the latest SFA export that
        listed the licence. An older one means SFA no longer gives it a grade there
        (e.g. "Not Under SAFE"), so there is no current grade."""
        if licensee is None or not licensee.grades:
            return None
        newest = _history(licensee)[0]
        if licensee.last_seen_snapshot and newest.snapshot_date < licensee.last_seen_snapshot:
            return None
        return newest

    @staticmethod
    def passes_threshold(stall: Stall, threshold: SafeGradeValue | None) -> bool:
        """True if the stall's current grade ranks at or above the minimum (FR-2.5, FR-5.4).
        With no minimum every stall passes; with one, a stall with no current grade (or no
        licensee) fails, so ungraded stalls are never shown to someone who asked for a minimum."""
        if threshold is None:
            return True
        g = HygieneController.current_grade(stall.licensee)
        return g is not None and GRADE_RANK[g.grade_value] >= GRADE_RANK[threshold]

    @staticmethod
    def summary(stalls: Iterable[Stall]) -> GradeSummary:
        """How many of a centre's Cooked Food stalls (FR-1.6) hold each current grade."""
        counts = {"A": 0, "B": 0, "C": 0, "NEW": 0, "ungraded": 0}
        for s in stalls:
            if s.stall_type != StallType.COOKED_FOOD:
                continue
            g = HygieneController.current_grade(s.licensee)
            counts[g.grade_value.value if g else "ungraded"] += 1
        return GradeSummary(**counts)

    @staticmethod
    def track_record(stall: Stall) -> TrackRecordOut:
        """Built from the snapshots, not stored (FR-2.3, FR-2.4). SFA only publishes a
        "suspended in the past year" flag, not demerit points, so total_demerit_points stays null."""
        lic = stall.licensee
        if lic is None:
            return TrackRecordOut(suspended_past_year=False, grade_history=[], last_updated=None)
        return TrackRecordOut(
            suspended_past_year=lic.licence_status == "suspended",
            grade_history=[SafeGradeOut.model_validate(g) for g in _history(lic)],
            last_updated=lic.last_seen_snapshot,
        )
```

## B.1 Exercise 3.3.3: our prompt

> This is our SC2006 group project, CleanCravings (FastAPI backend in backend/, React frontend in frontend/). Please review the code quality of the backend (backend/app and backend/etl).
>
> 1. Give us the five refactorings you think would most improve it, ranked, each with the reason and the risk.
> 2. Then carry out only your top-ranked refactoring. Keep the behaviour exactly the same and don't change any tests.
>
> You can't run commands in this session, so explain how you made sure the behaviour is unchanged. We will run the test suite afterwards.

## B.2 The agent's reply

I've ranked five refactorings below and done the top one. I couldn't run anything here, so the test suite hasn't been run yet.

### 1. The five refactorings, ranked

**1. Define each stall-filtering rule once (done, details below).**

- **Why:** the minimum-grade rule (FR-2.5) was written out three times. `HygieneController.passes_threshold` had it, the centre list kept its own grade-number comparison, and the recommendations kept a third copy with a comment saying "Same rule as…". The halal check and the "suspended" flag were each written twice. Your Lab 4 report (`docs/lab4/lab4_test_cases.html:46`) says `passes_threshold()` is what the centre list and recommendations use, but they never called it. So the 16 black-box tests only covered one of the three copies.
- **Risk:** low. Each change swaps code for an equivalent call. The only change in shape is in-memory cached data that nothing outside the controller reads.

**2. Share the centre query and the Cooked Food filter.**

- **Why:** `_centre_query()` and `_cooked()` in `hawker_centre_controller.py` are copied by hand into `recommendation_controller.py:47-62`. The stall loading options are repeated in `search` and `stall_detail`. Also, `centre_detail` and `stall_detail` call `hdp_counts(db)`, which counts every centre in the database just to read one number.
- **Risk:** low. If the shared query loads less up front than the copy did, you'd get many extra database queries (much slower), but the results wouldn't change.

**3. Split up `RecommendationController.recommend` (about 100 lines).**

- **Why:** loading the user's data, the filters, six counters, the scoring weights and the per-centre limit are all in one loop. The scoring is the logic you're most likely to tune, and right now it can't be tested on its own. Pull out a small user-context object, a `_score(...) -> (score, reasons)` function and a `_cap_per_centre` function.
- **Risk:** medium. The order of the reasons, the floating-point sums and the tie-breaks all have to stay exactly the same. Write unit tests that pin the current output before moving anything.

**4. Move business logic out of `routers/public.py`.**

- **Why:** the router works out freshness and staleness, picks the default dish profile, decides the halal default and calls OneMap directly. Your class diagrams describe routers as a thin layer over the controllers. The "SQLite drops the timezone" fix also exists twice (`user_controllers._utc` and an inline copy in `freshness`).
- **Risk:** low to medium. Watch for import cycles. Tests that patch `httpx` will have to patch it in the new module, which means editing tests, so do that step on its own.

**5. Cut the per-row database queries in the ETL scripts.**

- **Why:** `sfa_import.load` runs up to three queries for every row (licensee, stall, grade). That's tens of thousands of round trips per full export against Supabase. Load them all up front into dictionaries. Smaller fixes alongside it: `seed_reference.load` reads the rules CSV twice, and `nea_sync.parse_row` repeats the closure-reason logic.
- **Risk:** medium. Sessions are created with `autoflush=False`, so today a licence that appears twice in one export probably fails when the data is saved. With dictionaries, the second row would quietly update the first instead, which changes behaviour. Add a test for duplicate rows first.

### 2. What I changed (#1)

| File | Change |
|---|---|

| | |
|---|---|
| `hygiene_controller.py` | New `grade_passes(grade, threshold)` holds the one copy of the rule. `passes_threshold` now calls it. New `suspended_past_year(licensee)`, also used by `track_record`. |
| `hawker_centre_controller.py` | The cache stores each centre's best grade instead of its grade number. The centre list filter calls `grade_passes`. |
| `recommendation_controller.py` | Uses `grade_passes` and `confirmed_halal` instead of its own copies. The `GRADE_RANK` import is gone. |
| `presenters.py` | Uses `suspended_past_year`. `confirmed_halal` now also accepts a `StallSummary`. |

No tests, API fields or database queries changed.

### How I checked the behaviour is the same

I compared old and new code case by case:

- `passes_threshold`: it still returns `True` straight away when no minimum is set, so it doesn't look up the grade in that case, exactly as before. Otherwise it does the same two checks in the same order: no grade means hidden, then grade numbers are compared with `>=`.
- **Centre list:** before, it skipped a centre when `min_grade and best_rank < GRADE_RANK[min_grade]`, where `best_rank` was 0 if the centre had no graded stalls.
  - No minimum set: neither version skips the centre.
  - Minimum set, no graded stalls: old compares `0 < rank`, which is true, so skipped. New: `grade_passes(None, min)` is `False`, so skipped.
  - Minimum set, best grade `g`: each grade has its own distinct number, so the best grade has the highest number and both versions compare `GRADE_RANK[g] >= GRADE_RANK[min]`.
- **Recommendations:** before, it skipped when `threshold and (grade is None or rank < min)`. The minimum is a string enum and never empty, so `threshold` is truthy exactly when it isn't `None`. That makes the old condition the exact opposite of `grade_passes`. `s.halal is not True` is literally `not confirmed_halal(s)`, and the counters still go up on the same branches.
- **Suspended flag:** the same expression, moved into one function.
- **Imports:** I checked the import chain for cycles and confirmed everything is imported where it's used (`SafeGradeValue` was already imported in the centre controller).

Tests that cover these paths: the 16 `test_lab4.py::test_blackbox_passes_threshold` cases, plus the `min_grade`, halal and `excluded_by_hygiene` checks in `test_api.py`.

One gap: no existing test checks which centres `/api/centres?min_grade=…` returns; the only test there just checks a 422 for an invalid grade. That path is the one I reasoned through most carefully above. When you add tests next, one comparing that list against `passes_threshold` on each centre's stalls would cover it.

## B.3 The changes it made

```
diff -u -x __pycache__ original/backend/app/controllers/hawker_centre_controller.py refactored/backend/app/controlle
rs/hawker_centre_controller.py
--- original/backend/app/controllers/hawker_centre_controller.py        2026-11-05 23:34:55.000000000 +0800
+++ refactored/backend/app/controllers/hawker_centre_controller.py      2026-11-05 23:47:13.468803000 +0800
@@ -71,19 +71,19 @@
         return _photos[centre_id]

     @classmethod
-    def _cached_summaries(cls, db: Session, today: date) -> list[tuple[CentreSummary, int, str]]:
-        """Centre summaries without distance, plus the best current grade rank among each
+    def _cached_summaries(cls, db: Session, today: date) -> list[tuple[CentreSummary, SafeGradeValue | None, str]]:
+        """Centre summaries without distance, plus the best current grade among each
         centre's Cooked Food stalls and the text that search matches against. Cached (see
         controllers/cache.py), so each request only works out distance (NFR-1)."""

-        def build() -> list[tuple[CentreSummary, int, str]]:
+        def build() -> list[tuple[CentreSummary, SafeGradeValue | None, str]]:
             counts = cls.hdp_counts(db)
             rows = []
             for c in db.scalars(_centre_query()).all():
                 grades = (HygieneController.current_grade(s.licensee) for s in _cooked(c.stalls))
-                best_rank = max((GRADE_RANK[g.grade_value] for g in grades if g is not None), default=0)
+                best = max((g.grade_value for g in grades if g is not None), key=GRADE_RANK.__getitem__, default=No
ne)
                 haystack = f"{c.centre_name} {c.street_address or ''} {c.postal_code or ''}".lower()
-                rows.append((centre_summary(c, today, None, counts.get(c.id, 0)), best_rank, haystack))
+                rows.append((centre_summary(c, today, None, counts.get(c.id, 0)), best, haystack))
             return rows

         return cached("centre_summaries", db, today, build)
@@ -101,13 +101,14 @@
     ) -> list[CentreSummary]:
         out: list[CentreSummary] = []
         needle = (q or "").strip().lower()
-        for base, best_rank, haystack in cls._cached_summaries(db, today):
+        for base, best_grade, haystack in cls._cached_summaries(db, today):
             if region and base.region != region:
                 continue
             if needle and needle not in haystack:
                 continue
-            # Same result as passes_threshold() on each Cooked Food stall (FR-2.5)
-            if min_grade and best_rank < GRADE_RANK[min_grade]:
+            # A centre is listed if at least one Cooked Food stall meets the minimum (FR-2.5),
+            # i.e. if its best one does
+            if not HygieneController.grade_passes(best_grade, min_grade):
                 continue
             dist = round(haversine_km(origin[0], origin[1], base.latitude, base.longitude), 2) if origin else None
             if radius_km and dist is not None and dist > radius_km:
diff -u -x __pycache__ original/backend/app/controllers/hygiene_controller.py refactored/backend/app/controllers/hyg
iene_controller.py
--- original/backend/app/controllers/hygiene_controller.py      2026-11-05 23:34:55.000000000 +0800
+++ refactored/backend/app/controllers/hygiene_controller.py    2026-11-05 23:46:57.758932800 +0800
@@ -20,14 +20,26 @@
             return None  # latest SFA export had no SAFE grade for this licence
         return latest

+    @staticmethod
+    def grade_passes(grade: SafeGradeValue | None, threshold: SafeGradeValue | None) -> bool:
+        """The Hygiene Threshold rule (FR-2.5) on a grade value. Every list that filters by a
+        minimum grade (centre list, centre page, recommendations) goes through this."""
+        if threshold is None:
+            return True
+        if grade is None:
+            return False  # can't show an ungraded stall to someone who set a minimum
+        return GRADE_RANK[grade] >= GRADE_RANK[threshold]
+
     @classmethod
     def passes_threshold(cls, stall: Stall, threshold: SafeGradeValue | None) -> bool:
         if threshold is None:
             return True
         g = cls.current_grade(stall.licensee)
-        if g is None:
```

```
-            return False  # can't show an ungraded stall to someone who set a minimum
-        return GRADE_RANK[g.grade_value] >= GRADE_RANK[threshold]
+        return cls.grade_passes(g.grade_value if g else None, threshold)
+
+    @staticmethod
+    def suspended_past_year(licensee: Licensee | None) -> bool:
+        return bool(licensee and licensee.licence_status == "suspended")

     @classmethod
     def summary(cls, stalls: Iterable[Stall]) -> GradeSummary:
@@ -49,7 +61,7 @@
             if not changes or changes[-1].grade_value != g.grade_value:
                 changes.append(g)
         return TrackRecordOut(
-            suspended_past_year=bool(stall.licensee and stall.licensee.licence_status == "suspended"),
+            suspended_past_year=cls.suspended_past_year(stall.licensee),
             grade_history=[SafeGradeOut.model_validate(g) for g in reversed(changes)],
             last_updated=stall.licensee.last_seen_snapshot if stall.licensee else None,
         )
diff -u -x __pycache__ original/backend/app/controllers/presenters.py refactored/backend/app/controllers/presenters.
py
--- original/backend/app/controllers/presenters.py      2026-11-05 23:34:55.000000000 +0800
+++ refactored/backend/app/controllers/presenters.py    2026-11-05 23:47:02.808633300 +0800
@@ -19,7 +19,7 @@
         cuisine=stall.cuisine,
         price_range=stall.price_range,
         safe_grade=SafeGradeOut.model_validate(g) if g else None,
-        suspended_past_year=bool(stall.licensee and stall.licensee.licence_status == "suspended"),
+        suspended_past_year=HygieneController.suspended_past_year(stall.licensee),
         hdp_participant=stall.hdp is not None,
         hdp_category=stall.hdp.participation_category if stall.hdp else None,
         hdp_match_method=stall.hdp.match_method if stall.hdp else None,
@@ -31,7 +31,7 @@
     return bool(profile and "HALAL" in (profile.dietary_restrictions or []))


-def confirmed_halal(stall: Stall) -> bool:
+def confirmed_halal(stall: Stall | StallSummary) -> bool:
     # Only an explicit True counts. None means the stall hasn't published it, not that it's non-halal
     return stall.halal is True

diff -u -x __pycache__ original/backend/app/controllers/recommendation_controller.py refactored/backend/app/controll
ers/recommendation_controller.py
--- original/backend/app/controllers/recommendation_controller.py       2026-11-05 23:34:55.000000000 +0800
+++ refactored/backend/app/controllers/recommendation_controller.py     2026-11-05 23:47:19.727499600 +0800
@@ -15,8 +15,8 @@
 from ..schemas import CentreStatus, RecommendationItem, RecommendationOut, StallSummary
 from .cache import cached
 from .closure_controller import ClosureController
-from .hygiene_controller import GRADE_RANK
-from .presenters import stall_summary, wants_halal
+from .hygiene_controller import HygieneController
+from .presenters import confirmed_halal, stall_summary, wants_halal

 DISCLAIMER = (
     "Suggestions are ordered using official SFA, NEA and HPB data plus your preferences. "
@@ -104,12 +104,11 @@
                 continue
             for s in c.stalls:
                 grade = s.safe_grade.grade_value if s.safe_grade else None
-                # Same rule as HygieneController.passes_threshold: no current grade fails any minimum
-                if threshold and (grade is None or GRADE_RANK[grade] < GRADE_RANK[threshold]):
+                if not HygieneController.grade_passes(grade, threshold):
                     excluded_hygiene += 1
                     continue
                 if halal_only:
-                    if s.halal is not True:  # as presenters.confirmed_halal: None means unknown
+                    if not confirmed_halal(s):
                         excluded_halal += 1
                         continue
                     halal_confirmed += 1
```